Buku Pintar Vibe Coder

Belajar membangun aplikasi dengan bantuan AI secara praktis, kritis, dan bertanggung jawab

Ferry

Edisi 1 · 2026

Halaman Hak Cipta dan Penggunaan

© 2026 Ferry. Seluruh hak cipta dilindungi. Dilarang memperbanyak, mendistribusikan ulang, atau menjual ulang sebagian maupun seluruh isi buku ini tanpa izin tertulis dari penulis. Kutipan singkat untuk keperluan ulasan atau pembelajaran diperbolehkan dengan menyebutkan sumber.

Cara mengutip: Ferry. (2026). *Buku Pintar Vibe Coder*, Edisi 1.

Sifat informasi. Buku ini bersifat edukatif dan berbagi pengalaman. Buku ini bukan nasihat hukum, audit keamanan, atau nasihat keuangan. Penulis tidak bertanggung jawab atas kerugian yang timbul dari penerapan isi buku tanpa pemeriksaan, pengujian, dan pertimbangan profesional yang sesuai.

Merek dagang. GitHub, Vercel, Supabase, Google (Google Sheets dan Google Apps Script), Next.js, React, Vite, WordPress, Shopify, Canva, Gemini, dan Claude adalah milik pemiliknya masing-masing. Buku ini tidak berafiliasi dengan, tidak disponsori oleh, dan tidak diendorse oleh pemilik merek tersebut.

Kontak koreksi dan masukan: peyaje531@gmail.com

Catatan Penting Sebelum Membaca

Harga, kuota, dan layanan dapat berubah. Harga, kuota, paket gratis, fitur, dokumentasi, dan ketentuan layanan pada alat yang dibahas dapat berubah kapan saja. Buku ini tidak menetapkan angka yang berlaku saat ini. Periksa sumber resmi pada tanggal keputusan Anda dan catat hasil pemeriksaannya. Daftar sumber resmi ada di Lampiran G.

Buku ini memisahkan tiga status pengetahuan:

- **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis):** pengalaman operasional yang benar-benar dilakukan penulis. Penggunaan operasional bukan bukti keamanan menyeluruh, pengujian terstruktur, pemulihan bencana, performa pada skala besar, atau kesiapan komersial.
- **Contoh Ilustrasi untuk Belajar:** gambaran konseptual untuk membantu memahami materi. Bukan bukti hasil nyata. Setiap contoh tanpa catatan pengalaman penulis harus dianggap sebagai contoh ilustrasi.
- **Belum Diuji / Rencana:** informasi atau langkah yang belum memiliki bukti pengujian yang dapat ditunjukkan. Tidak boleh dibaca sebagai hasil yang sudah terbukti. Daftarnya ada di Lampiran F.

Daftar Isi

Catatan Penting Sebelum Membaca **ii**

Pengantar Penulis **1**
- Mengapa buku ini ditulis . . . 1
- Identitas dan pengalaman penulis . . . 1
- Batas pengalaman . . . 2
- Catatan transparansi penyusunan . . . 2
- Kegagalan nyata yang perlu diceritakan . . . 3
- Yang benar-benar berjalan di usaha penulis . . . 4
- Bagaimana buku ini diuji . . . 4
- Hal yang tidak dijanjikan . . . 5
- Cara membaca buku ini . . . 5

Bab 1 — Cara Berpikir, Batas AI, dan Kapan Harus Berhenti **6**
- 1.1 Apa itu vibe coder? . . . 6
- 1.2 Pembagian tugas yang sehat . . . 6
- 1.3 Tiga dasar yang perlu dipahami . . . 7
- 1.4 Pelajaran dari sistem yang terlalu rumit . . . 7
- 1.5 Kapan harus memanggil programmer profesional? . . . 8
- 1.6 Latihan singkat . . . 9

Bab 2 — Peta Teknologi, Dipilih Sesuai Kebutuhan **10**
- 2.1 Perumpamaan usaha . . . 10
- 2.2 Kelompok pilihan . . . 10
- 2.3 Memahami kecocokan, bukan mencari alat terbaik mutlak . . . 11
- 2.4 Empat pertanyaan pemilihan . . . 12
- 2.5 Pertanyaan pemeriksaan sebelum memilih . . . 12
- 2.6 Latihan singkat . . . 12

Bab 3 — Anatomi Proyek: Satu Contoh Stack Next.js **13**

3.1 Mengapa isi proyek perlu dipahami? 13
3.2 Peta folder yang umum 14
3.3 Nama folder dapat berbeda 14
3.4 Membaca alur sederhana 15
3.5 Latihan membaca proyek sederhana 15
3.6 Pola hibrida: tampilan di Vercel, data di Google Sheets lewat GAS 16

Bab 4 — Bekerja dengan AI Secara Terstruktur 18
4.1 Mulai dari spesifikasi, bukan dari permintaan kabur 18
4.2 Kerjakan langkah kecil 18
4.3 Berikan konteks yang cukup 19
4.4 Contoh prompt buruk dan prompt lebih baik 19
4.5 Minta penjelasan, bukan hanya hasil 19
4.6 Catatan proyek 20
4.7 Menggunakan lebih dari satu AI 20
4.8 Prompt nyata dan pengalaman penulis 20
4.9 Latihan menulis prompt 21

Bab 5 — Memeriksa dan Mencurigai Hasil AI 22
5.1 Mengapa hasil AI perlu dicurigai? 22
5.2 Jenis kesalahan yang sering muncul 22
5.3 Pemeriksaan dokumentasi dan versi 22
5.4 Baca perubahan sebelum menjalankan 23
5.5 Typecheck, lint, tes, dan uji manual 23
5.6 Kasus uang dan persediaan 23
5.7 Tampil benar tidak sama dengan benar 23
5.8 Laporan yang jujur 24
5.9 Daftar tanda bahaya 24
5.10 Latihan pemeriksaan 24
5.11 Temuan lapangan dan pelajaran 24

Bab 6 — Keamanan Dasar dan Data Pelanggan 26
6.1 Batas pembahasan 26
6.2 Rahasia di lingkungan proyek 26
6.3 Login dan hak akses 27

6.4 Memisahkan data antar toko atau pengguna 27
6.5 Validasi input 27
6.6 Penyimpanan kata sandi 27
6.7 Minimalkan data pelanggan 27
6.8 Kesadaran dasar tentang pelindungan data pribadi 28
6.9 Catatan teknis per platform 28
6.10 Status keamanan aplikasi penulis 29
6.11 Checklist sebelum rilis 29

Bab 7 — Version Control dan Cadangan **30**
7.1 Git dan GitHub dengan perumpamaan 30
7.2 Buat titik aman sebelum perubahan besar 30
7.3 Cabang untuk pekerjaan terpisah 30
7.4 Rollback langkah demi langkah 31
7.5 Cadangan data terpisah dari kode 31
7.6 Yang tidak boleh masuk repository 31
7.7 Latihan pemulihan 31

Bab 8 — Biaya dan Pilihan Penyimpanan Data **33**
8.1 Biaya bukan hanya tagihan bulanan 33
8.2 Pantau pemakaian dan buat batas 33
8.3 Kepemilikan akun dan data 34
8.4 Google Sheets + GAS 34
8.5 Database cloud seperti Supabase 34
8.6 Cara membandingkan secara adil 35
8.7 Catatan sumber 36
8.8 Latihan keputusan 36

Bab 9 — Proyek Perdana dari Awal sampai Rilis **37**
9.1 Pilih proyek kecil dan nyata 37
9.2 Sepuluh langkah kerja 37
9.3 Contoh spesifikasi ringkas 39
9.4 Sebelum menyebutnya rilis 39

Bab 10 — Setelah Rilis, Pantau dan Rawat **40**
10.1 Rilis adalah awal perawatan 40

10.2 Pemantauan rutin 40
10.3 Saat error terjadi di jam operasional 40
10.4 Prosedur manual sementara 41
10.5 Mencari akar masalah 41
10.6 Perawatan berkala 41
10.7 Laporan pengguna 41
10.8 Rencana cadangan manual 41

Bab 11 — Serah Terima ke Klien (Rancangan) 42
11.1 Kepemilikan akun 42
11.2 Dokumentasi minimum 42
11.3 Pelatihan pemakai dan pemilik 43
11.4 Batas tanggung jawab dan dukungan 43
11.5 Kapan klien perlu beralih ke programmer profesional? . . . 43
11.6 Bukti serah terima 43

Penutup 44

Tentang Komunitas dan Koreksi Naskah 45

Lampiran A — Glosarium Istilah Teknis 46

Lampiran B — Checklist Keamanan Sebelum Rilis 48

Lampiran C — Checklist Pemeriksaan Hasil AI dan Tanda Bahaya 49

Lampiran D — Templat Prompt untuk Belajar 50

Lampiran E — Catatan Versi Buku 51

Lampiran F — Daftar Terbuka Hal-Hal yang Belum Diuji Penulis 52

Lampiran G — Sumber Resmi dan Tanggal Rujukan 54

Pengantar Penulis

Mengapa buku ini ditulis

Buku ini ditulis dari sudut pandang pelaku usaha yang membangun aplikasi dengan bantuan AI, bukan dari sudut pandang programmer profesional. AI dapat membantu menerjemahkan kebutuhan usaha menjadi rancangan dan perubahan teknis, tetapi AI bukan pengganti tanggung jawab manusia. Pemilik proyek tetap perlu memahami tujuan, memeriksa hasil, menjaga data, membuat cadangan, dan tahu kapan harus berhenti.

Buku ini tidak menjanjikan bahwa pembaca akan menjadi programmer profesional secara instan. Tujuannya lebih sederhana dan lebih berguna: membantu pembaca memulai proyek kecil dengan cara yang tertib, memahami apa yang sedang dibangun, mengenali risiko, dan mengambil keputusan yang lebih sadar.

Identitas dan pengalaman penulis

Latar belakang usaha. Pelaku usaha kuliner Bakso Len di Lubuklinggau dan Curup.

Pengalaman menggunakan AI. Penulis mempelajari dan mempraktikkan vibe coding secara intensif melalui proses bertahap: memahami kebutuhan usaha, berdiskusi dengan berbagai model AI termasuk Gemini dan Claude, mencoba perubahan, memeriksa hasil, serta mencatat kesalahan dan pembelajaran yang muncul. Buku ini adalah intisari dari jurnal perjalanan belajar penulis dari nol. Karena teknologi dan layanan terus berkembang, buku ini diperlakukan sebagai dokumen hidup (*living document*) yang diperbarui ketika ditemukan koreksi, bukti baru, atau perubahan yang relevan.

Masalah yang mendorong penulisan. Kebutuhan nyata di usaha penulis sendiri, yaitu pencatatan kasir di cabang Curup dan pengelolaan stok di Lubuklinggau. Penulis juga ingin belajar bersama orang lain melalui Komunitas Ngulik Digital yang penulis inisiasi.

Batas pengalaman

Penulis pernah menggunakan paket gratis GitHub, Vercel, dan Supabase pada empat proyek awal. Namun, penggunaan Supabase tersebut tidak menjadi bukti bahwa kombinasi Next.js dan Supabase pada contoh buku ini pernah diterapkan dan diuji sebagai satu sistem. Contoh Next.js dan Supabase di buku ini bersifat ilustrasi. Pola yang dicatat sebagai pengalaman penulis adalah GitHub dan Vercel untuk tampilan depan, dengan Google Sheets dan Google Apps Script (GAS) untuk penyimpanan data.

Catatan transparansi penyusunan

Naskah ini disusun melalui tanya-jawab dan kolaborasi interaktif antara penulis, Ferry, dan asisten AI: Gemini, Claude, dan Canva AI. Gemini dan Claude digunakan melalui aplikasi masing-masing untuk membantu perumusan struktur, analisis logika, penjabaran materi, dan peninjauan naskah. Canva AI digunakan untuk membantu penyuntingan dan penyusunan tata letak di platform Canva.

Penulis tetap menentukan pengalaman mana yang benar-benar terjadi, memeriksa klaim, menilai hasil, dan bertanggung jawab atas keputusan yang dicantumkan. Keluaran AI tidak dianggap sebagai bukti dengan sendirinya. Penulis membuka diri bagi siapa saja yang ingin memberikan koreksi, masukan, atau kritik agar buku ini semakin relevan, valid, dan bermanfaat.

Kegagalan nyata yang perlu diceritakan

> **Status pengetahuan: Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis).**

1. Terlalu rumit di proyek POS. Saya terlalu bersemangat meminta AI menambah fitur yang tidak dibutuhkan di lapangan. Setelah ditinjau ulang, saya harus memangkas puluhan berkas yang tidak perlu agar aplikasi kembali menjadi POS yang sederhana, ringan, dan fokus pada transaksi utama.

2. Tampilan iframe pada Google Apps Script. Aplikasi Stok Bakso Len semula saya bangun utuh di dalam GAS. Setelah dicoba, tampilan akhirnya menyisakan ruang kosong yang lebar di bagian bawah layar. Saya memindahkan tampilan depannya ke GitHub dan Vercel, dan tetap memakai Google Sheets dengan GAS untuk data.

3. Kuota pemeriksaan otomatis habis. Pada satu proyek, kuota pemeriksaan otomatis di GitHub Actions habis. Saya kurang memahami rincian teknisnya. Yang saya ketahui, lewat bantuan AI pernah dibuatkan sekitar empat atau lima berkas pemeriksaan otomatis berformat `.yml`, dan semuanya berjalan setiap kali aplikasi diterbitkan (di-*deploy*). Kuota layanan saat itu terpakai habis. Saya meminta semuanya dinonaktifkan, lalu proyek bisa diperbaiki dan disempurnakan lagi secara bertahap.

Pelajarannya: setiap tambahan otomatisasi dari AI dapat memiliki ongkos dan batas pemakaian. Tanyakan apa yang dijalankan, kapan dijalankan, dan berapa pemakaiannya. Angka 2.000 menit per bulan adalah catatan historis dari pengalaman saya saat itu, bukan klaim kuota yang berlaku sekarang. Saya tidak mencatat apakah repository saat itu publik atau privat, sehingga angka tersebut tidak boleh dipakai sebagai rekomendasi.

Yang benar-benar berjalan di usaha penulis

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis)**

- Penggunaan paket gratis GitHub, Vercel, dan Supabase pada empat proyek awal, pada periode penggunaan tersebut. Ini bukan jaminan paket atau batas layanan yang sama masih berlaku.
- Pola hibrida GitHub + Vercel (tampilan depan) dengan Google Sheets + GAS (penyimpanan data) pada Aplikasi Stok Bakso Len, untuk mengatasi masalah iframe dan ruang kosong pada GAS murni.
- Operasional harian POS UMKM Juara oleh staf kasir di cabang Curup, dan Aplikasi Stok Bakso Len oleh staf pengelola stok di Lubuklinggau.
- Penggunaan dokumen acuan proyek (`agen.md`), skill, dan custom instructions di repositori untuk mengarahkan AI agar tidak asal menghapus kode.

Batas status ini: pengalaman operasional tersebut belum berarti bahwa seluruh aspek keamanan, hak akses, pemulihan, performa pada skala besar, atau kesiapan komersial telah dibuktikan.

Bagaimana buku ini diuji

POS UMKM Juara dipakai oleh staf kasir cabang Curup. Aplikasi Stok Bakso Len dipakai oleh staf pengelola stok di Lubuklinggau. Staf hanya menjalankan pekerjaan harian dan tidak mengubah data atau pengaturan. Bila ada kendala, mereka melapor kepada saya, lalu saya yang memperbaiki aplikasinya.

Semua aplikasi sudah saya uji coba, tetapi belum tuntas 100%, karena saya juga menjalankan usaha setiap hari dan mengerjakan beberapa proyek sekaligus. Hal-hal yang belum diuji dicatat terbuka di Lampiran F.

Hal yang tidak dijanjikan

- **Tidak menjanjikan keamanan mutlak.** Keamanan dasar mengurangi risiko umum, tetapi bukan jaminan bebas dari semua serangan.
- **Tidak menjanjikan biaya tetap.** Harga, kuota, paket gratis, dan kebijakan layanan dapat berubah.
- **Tidak menjanjikan hasil tanpa pemeriksaan.** Keluaran AI dapat keliru, tidak sesuai versi, atau tampak benar tetapi salah secara logika.
- **Tidak memberikan nasihat hukum.** Pembahasan data pelanggan dan aturan hanya kesadaran dasar; keputusan hukum perlu dibicarakan dengan pihak yang kompeten.

Cara membaca buku ini

Baca Bab 1 dan Bab 2 untuk memahami cara berpikir serta peta pilihan. Gunakan Bab 3 dan Bab 4 ketika mulai membaca proyek dan bekerja dengan AI. Jadikan Bab 5 sampai Bab 7 sebagai kebiasaan pemeriksaan, keamanan, dan pencadangan. Bab 8 membantu menimbang biaya. Bab 9 memberi alur proyek pertama, sedangkan Bab 10 dan Bab 11 membahas masa setelah rilis dan serah terima.

Gunakan checklist di Lampiran B dan C secara langsung. Jika menemukan istilah baru, buka Lampiran A. Catat setiap keputusan dan bagian yang belum diuji. Jangan melewati bagian yang membahas batasan hanya karena bagian itu tidak semenarik pembuatan tampilan.

Bab 1 — Cara Berpikir, Batas AI, dan Kapan Harus Berhenti

> **Status pengetahuan: Contoh Ilustrasi untuk Belajar.** Bab ini membahas kerangka berpikir umum, bukan catatan pengalaman lapangan penulis.

1.1 Apa itu vibe coder?

Vibe coder adalah orang yang membangun atau mengubah aplikasi dengan bantuan AI melalui percakapan, instruksi, dan pemeriksaan bertahap. Ia tidak harus memulai sebagai programmer. Namun, ia tetap bertanggung jawab atas masalah yang dipilih, data yang diproses, pengguna yang dilayani, dan keputusan untuk merilis atau menunda.

Istilah ini tidak berarti aplikasi dibuat hanya dengan perasaan atau tebakan. Vibe coding yang bertanggung jawab membutuhkan spesifikasi, konteks, perubahan kecil, pemeriksaan, pencatatan, pengujian, dan kemampuan untuk membatalkan perubahan.

AI membantu	Manusia memutuskan
Membuat draf tampilan, menjelaskan istilah, menyarankan struktur, menulis contoh, mencari kemungkinan penyebab error, dan mengubah instruksi menjadi langkah teknis.	Tujuan usaha, prioritas, arti data, hak akses, risiko, kebenaran hasil, keamanan, biaya, dan kapan perubahan harus dihentikan.

1.2 Pembagian tugas yang sehat

Jangan menyerahkan seluruh proyek kepada AI sebagai satu permintaan besar. Bagilah pekerjaan menjadi keputusan yang dapat dijelaskan. Misalnya, manusia menetapkan bahwa hanya pemilik toko yang boleh

menghapus barang; AI dapat membantu menerjemahkan aturan itu menjadi tampilan dan pemeriksaan, tetapi manusia tetap harus menguji apakah pengguna biasa benar-benar tidak dapat menghapusnya.

Wilayah	Tugas manusia	Bantuan AI
Tujuan	Menentukan masalah dan hasil yang diinginkan.	Membantu merumuskan pertanyaan dan pilihan.
Aturan usaha	Menentukan siapa boleh melakukan apa.	Membantu menulis skenario dan validasi.
Implementasi	Memilih perubahan yang disetujui.	Membuat draf dan menjelaskan dampak.
Verifikasi	Menguji dan menilai bukti.	Menyusun daftar tes dan kemungkinan masalah.
Rilis	Menilai risiko, cadangan, dan kesiapan.	Membantu membuat checklist.

1.3 Tiga dasar yang perlu dipahami

Alur data. Data biasanya masuk dari tampilan, diperiksa, dikirim ke bagian pemrosesan, disimpan, lalu ditampilkan kembali. Kesalahan dapat terjadi di setiap perpindahan. Nilai yang terlihat di layar belum tentu sudah tersimpan dengan benar.

Struktur proyek. Proyek memiliki tempat untuk tampilan, komponen yang dipakai berulang, aturan pemrosesan, koneksi layanan, dan pengaturan. Nama folder berbeda menurut kerangka aplikasi, tetapi gagasan pemisahan tanggung jawabnya tetap penting.

Keamanan. Keamanan bukan satu fitur tambahan. Ia mencakup rahasia, login, hak akses, validasi input, pemisahan data, pencadangan, dan cara merespons ketika sesuatu bocor atau gagal.

1.4 Pelajaran dari sistem yang terlalu rumit

Sistem yang terlalu rumit biasanya muncul ketika proyek menambah banyak fitur sebelum masalah awal dipahami. Setiap fitur menambah data, aturan, layar, kemungkinan error, dan kebutuhan perawatan.

AI dapat mempercepat penambahan tersebut, tetapi tidak otomatis membuat keseluruhan sistem lebih sederhana.

- **Mulai dari satu alur utama.** Pilih tindakan yang paling sering dan paling bernilai bagi pengguna.
- **Tunda fitur yang belum diperlukan.** Jangan menambah pembayaran, laporan kompleks, atau integrasi hanya karena bisa dibuat.
- **Catat asumsi.** Misalnya, "satu barang memiliki satu satuan" atau "hanya ada satu toko". Asumsi yang tidak dicatat mudah berubah menjadi bug.
- **Ukur kemampuan pemeliharaan.** Jika pemilik proyek tidak dapat menjelaskan cara menjalankan, mencadangkan, dan memperbaiki sistem, proyek mungkin sudah terlalu rumit.

1.5 Kapan harus memanggil programmer profesional?

Panggil programmer profesional ketika kesalahan dapat menimbulkan dampak besar, ketika sistem menyimpan data sensitif dalam skala berarti, ketika hak akses sulit dipastikan, ketika transaksi keuangan harus akurat, atau ketika aplikasi menjadi bagian penting dari operasional. Bantuan profesional juga layak dicari ketika error tidak dapat dijelaskan setelah pemeriksaan dasar, perubahan AI berulang kali merusak bagian lain, atau Anda tidak mampu membuat cadangan dan memulihkan sistem.

Berhenti sejenak jika Anda tidak tahu data apa yang berubah, tidak memiliki titik pemulihan, tidak dapat membedakan akun penguji dan akun nyata, atau merasa harus menerima jawaban AI karena tidak mampu memeriksanya.

> **Catatan penting.** Aplikasi POS yang mencatat transaksi harian membutuhkan ketelitian data dan pemeriksaan yang sesuai dengan risikonya. Penggunaan operasional tidak membuktikan bahwa keamanan, ketepatan transaksi, atau pemulihan setelah kegagalan sudah memadai. Sampai uji pemulihan dilakukan dan dicatat, cadangan tidak boleh dianggap terbukti dapat dipulihkan. Libatkan profesional bila risiko atau kemampuan pemeriksaan berada di luar kompetensi tim.

1.6 Latihan singkat

Pilih satu ide aplikasi. Tulis satu kalimat masalahnya, siapa pengguna utamanya, satu alur utama, data yang disimpan, dan risiko terbesar. Kemudian jawab: bagian mana yang dapat dibantu AI, bagian mana yang harus diputuskan manusia, dan kondisi apa yang membuat Anda meminta bantuan profesional?

Bab 2 — Peta Teknologi, Dipilih Sesuai Kebutuhan

> **Status pengetahuan: Contoh Ilustrasi untuk Belajar.** Bab ini menyajikan peta pilihan umum, bukan klaim pengalaman lapangan untuk setiap alat.

2.1 Perumpamaan usaha

Bayangkan aplikasi sebagai sebuah usaha. *Front-end* adalah toko dan meja kasir yang dilihat pelanggan. *Back-end* adalah ruang kerja yang memproses pesanan dan aturan. *Database* adalah gudang catatan. *Hosting* adalah tempat usaha tersebut beroperasi agar dapat diakses. *API* adalah loket atau kurir yang menghubungkan bagian-bagian berbeda. *Repository* adalah arsip perubahan dan bahan kerja proyek.

Perumpamaan ini membantu, tetapi tidak menggantikan pemahaman teknis. Tujuannya adalah memberi peta awal agar pembaca tidak memilih alat hanya karena namanya populer atau karena AI menyarankannya.

2.2 Kelompok pilihan

Kebutuhan	Pilihan	Cocok untuk	Batas yang perlu diingat
Desain dan no-code	Canva dan alat sejenis	Konsep visual, materi, prototipe, dan alur sederhana.	Tidak otomatis menjadi aplikasi operasional dengan aturan data yang lengkap.
Situs siap pakai	WordPress	Konten, profil usaha, blog, dan situs yang memanfaatkan ekosistem plugin.	Plugin, pembaruan, kompatibilitas, dan keamanan perlu dirawat.

Kebutuhan	Pilihan	Cocok untuk	Batas yang perlu diingat
Toko online siap pakai	Shopify	Penjualan online dengan fitur perdagangan yang telah disediakan.	Biaya, aturan platform, dan fleksibilitas mengikuti layanan yang dipilih.
Kerangka aplikasi	React + Vite	Tampilan aplikasi yang membutuhkan kendali lebih langsung atas struktur front-end.	Keputusan tentang back-end, routing, keamanan, dan deployment tetap harus dibuat.
Kerangka aplikasi	Next.js	Aplikasi web yang menggabungkan tampilan, routing, dan kemampuan pemrosesan sesuai kebutuhan.	Konsepnya lebih luas; pemeliharaan dan pemahaman aturan tetap diperlukan.
Alur kerja	GitHub dan Vercel	Menyimpan perubahan, meninjau, dan menerbitkan aplikasi web.	Pengaturan akses, rahasia, biaya, dan perubahan layanan perlu dipantau.
Penyimpanan data	Google Sheets + GAS	Data kecil, pengguna sedikit, dan laporan sederhana.	Kuota harian, penulisan bersamaan, serta pengaturan akses yang terbatas.
Penyimpanan data	Database cloud seperti Supabase	Banyak pengguna, data terstruktur, dan aturan akses lebih rinci.	Biaya dapat naik, paket gratis dapat berubah, dan aturan akses perlu dipahami.

2.3 Memahami kecocokan, bukan mencari alat terbaik mutlak

Canva dapat sangat baik untuk menyusun rancangan layar atau materi komunikasi, tetapi rancangan visual bukan pengganti sistem yang menyimpan data dan menjalankan aturan. WordPress dan Shopify dapat mempercepat situs atau toko, tetapi pengguna perlu menerima batasan dan pola kerja platform tersebut. React + Vite memberi kendali pada front-end, sedangkan Next.js dapat menjadi pilihan untuk aplikasi web yang membutuhkan struktur lebih menyeluruh. Pilihan hosting dan

database tidak boleh dipisahkan dari kemampuan orang yang akan memeliharanya.

2.4 Empat pertanyaan pemilihan

1. **Siapa penggunanya?** Satu orang, beberapa staf, pelanggan umum, atau banyak organisasi? Jumlah dan variasi pengguna memengaruhi login, hak akses, dan kebutuhan dukungan.
2. **Seberapa besar volume data?** Pertimbangkan jumlah baris, frekuensi perubahan, ukuran berkas, serta kebutuhan pencarian dan laporan. "Sedikit sekarang" belum tentu berarti sedikit dalam beberapa bulan.
3. **Siapa pemeliharanya?** Pemilik usaha, staf internal, agensi, atau programmer profesional? Alat yang cocok harus dapat dipahami oleh orang yang benar-benar akan merawatnya.
4. **Berapa anggarannya?** Masukkan biaya langganan, domain, penyimpanan, layanan email, dukungan, waktu pemeliharaan, dan biaya perpindahan jika layanan berubah.

2.5 Pertanyaan pemeriksaan sebelum memilih

- Apakah saya dapat mengekspor data dalam bentuk yang berguna?
- Apa yang terjadi jika layanan berubah harga atau berhenti tersedia?
- Siapa yang memiliki akun dan data?
- Bagaimana saya memulihkan proyek jika perubahan terakhir bermasalah?
- Apakah aturan akses dapat diuji dan dijelaskan?
- Apakah paket yang saya pakai mengizinkan penggunaan untuk keperluan usaha (komersial)?

2.6 Latihan singkat

Buat tabel keputusan untuk satu kebutuhan usaha. Isi empat kolom: pilihan, alasan cocok, batas, dan pertanyaan yang belum terjawab. Jangan memilih sebelum menulis batasnya. Jika jawaban tentang volume data, pemelihara, atau anggaran masih kabur, tandai sebagai belum diputuskan dan jangan mengisinya dengan asumsi.

Bab 3 — Anatomi Proyek: Satu Contoh Stack Next.js

> **Status pengetahuan: Contoh Ilustrasi untuk Belajar.** Struktur folder, arsitektur Next.js, dan Supabase di Bab 3 dan Bab 9 adalah contoh konseptual untuk pembelajaran, bukan laporan hasil pengujian penulis. Lihat "Batas pengalaman" di Pengantar. Pembaca tetap harus menguji konfigurasi, versi, keamanan, dan kecocokannya pada proyek masing-masing. Bagian 3.6 membahas pola hibrida yang lebih dekat dengan pengalaman penulis.

3.1 Mengapa isi proyek perlu dipahami?

AI dapat menulis perubahan yang tampak meyakinkan tanpa memahami seluruh tujuan proyek. Jika pembaca mengenali tempat tampilan, aturan, koneksi data, dan pengaturan, ia dapat menilai perubahan dengan lebih baik. Tujuannya bukan menghafal semua detail, melainkan mengetahui bagian mana yang mungkin terdampak.

Bagian 3.1 sampai 3.5 menggunakan satu contoh stack: Next.js untuk aplikasi web, Vercel sebagai contoh tempat penerbitan, dan Supabase sebagai contoh database cloud.

3.2 Peta folder yang umum

Bagian	Peran umum	Pertanyaan pemeriksaan
`public`	Menyimpan berkas yang dapat disajikan langsung, seperti ikon atau gambar yang memang dipublikasikan.	Apakah berkas ini aman untuk dilihat umum?
`components`	Potongan tampilan yang digunakan kembali, misalnya tombol, kartu, atau formulir.	Apakah perubahan komponen memengaruhi banyak layar?
`app` atau `pages`	Tempat halaman, rute, dan alur tampilan aplikasi sesuai pola kerangka yang digunakan.	Halaman mana yang dapat diakses pengguna tertentu?
`lib`	Fungsi pendukung, aturan bersama, atau pembungkus koneksi layanan.	Apakah aturan penting hanya ada di tampilan atau juga diperiksa di server?
`api`	Jalur komunikasi untuk menerima permintaan dan mengirim respons, bila proyek menggunakannya sebagai bagian terpisah.	Input apa yang diterima dan siapa yang boleh mengaksesnya?
`package.json`	Daftar paket, perintah kerja, dan informasi proyek.	Paket apa yang dipakai dan kapan terakhir diperiksa versinya?
`.env`	Tempat lokal untuk nilai konfigurasi atau rahasia yang tidak seharusnya ditulis langsung dalam proyek.	Apakah isinya tidak dibagikan dan tidak masuk repository?
`.gitignore`	Daftar berkas yang tidak perlu atau tidak boleh dicatat dalam repository.	Apakah rahasia, hasil build, dan berkas lokal sudah dikecualikan?

3.3 Nama folder dapat berbeda

Kerangka aplikasi dan versi proyek dapat menggunakan nama serta susunan folder yang berbeda. Sebagian proyek Next.js menggunakan `app`, sebagian menggunakan `pages`, dan proyek lain dapat memindahkan

fungsi pendukung ke nama folder berbeda. Jangan menganggap nama folder sebagai hukum universal. Tanyakan kepada dokumentasi proyek dan baca hubungan antarberkasnya.

3.4 Membaca alur sederhana

1. Mulai dari halaman yang terlihat pengguna. Catat input, tombol, dan pesan hasilnya.
2. Cari komponen yang dipanggil halaman tersebut. Periksa apakah komponen digunakan di tempat lain.
3. Ikuti fungsi yang mengirim atau mengambil data. Catat apakah data hanya berubah di layar atau benar-benar disimpan.
4. Temukan konfigurasi layanan dan pastikan rahasia tidak tertulis langsung dalam berkas tampilan.
5. Baca perintah di `package.json` sebelum menjalankan langkah yang disarankan AI.

> **Catatan penting.** Berkas yang tampak kecil dapat berdampak luas. Perubahan pada komponen bersama, aturan akses, atau fungsi penyimpanan perlu diuji pada semua alur yang menggunakannya.

3.5 Latihan membaca proyek sederhana

Pilih proyek contoh yang tidak berisi data pelanggan nyata. Temukan folder atau berkas yang menjalankan fungsi berikut: menampilkan halaman, menerima input, menghubungkan penyimpanan, dan menyimpan pengaturan. Jelaskan dengan kata sendiri fungsi setiap bagian dalam empat kalimat. Jika tidak dapat menjelaskannya, jangan meminta AI menambah fitur besar; minta AI menjelaskan satu bagian pada satu waktu.

Tuliskan hasil latihan:

- Bagian tampilan dan rute utama: _____
- Komponen yang digunakan ulang: _____
- Fungsi pemrosesan atau koneksi data: _____
- Konfigurasi yang perlu dirahasiakan: _____

- Hal yang belum saya pahami: _____

3.6 Pola hibrida: tampilan di Vercel, data di Google Sheets lewat GAS

> **Status pengetahuan: Contoh Ilustrasi untuk Belajar.** Ini penjelasan konsep umum. Penulis memakai pola serupa pada Aplikasi Stok Bakso Len (lihat Pengantar), tetapi rincian teknis proyek penulis tidak dibahas di sini.

Alur umum.

1. Pengguna membuka tampilan depan yang diterbitkan di Vercel dari repository GitHub.
2. Tampilan mengirim permintaan ke alamat *web app* GAS.
3. GAS membaca atau menulis data di Google Sheets.
4. GAS mengirim balasan, lalu tampilan memperbarui layar.

Mengapa pola ini dipakai. GAS murni menampilkan halaman dalam bingkai (*iframe*), sehingga tata letak dapat menyisakan ruang kosong di layar. Memisahkan tampilan depan memberi kendali lebih atas tata letak, sementara Google Sheets tetap menjadi tempat data yang mudah dipahami pemilik usaha.

Pertanyaan keamanan yang perlu dijawab pada pola ini.

- Siapa yang dapat memanggil alamat web app? Agar dapat dipanggil dari browser, pengaturan akses sering dibuat longgar. Artinya, siapa pun yang mengetahui alamatnya dapat mencoba memanggilnya.
- Apakah pemeriksaan identitas dan peran dilakukan di sisi GAS, bukan hanya di tampilan?
- Skrip berjalan atas identitas siapa (“Execute as”)? Apa dampaknya jika alamat jatuh ke tangan orang lain?
- Apakah GAS memvalidasi input dan membatasi apa yang boleh diubah?
- Apakah ada salinan Google Sheets yang dapat dipulihkan jika data rusak atau terhapus?

Jangan menganggap alamat yang “tidak dipublikasikan” sebagai pengaman. Periksa dokumentasi resmi Google Apps Script untuk pilihan akses dan identitas eksekusi (Lampiran G).

Bab 4 — Bekerja dengan AI Secara Terstruktur

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis)** hanya berlaku untuk penggunaan dokumen acuan, custom instructions, dan skill pada bagian 4.8. Materi teknis lain dalam bab ini tetap perlu diperiksa dan tidak otomatis menjadi bukti pengujian menyeluruh.

4.1 Mulai dari spesifikasi, bukan dari permintaan kabur

Spesifikasi adalah catatan tentang masalah, pengguna, alur, data, aturan, dan hasil yang dianggap benar. Spesifikasi tidak perlu panjang. Satu halaman yang jelas lebih berguna daripada permintaan panjang yang mencampur tujuan, tampilan, database, dan rilis dalam satu paragraf.

Gunakan urutan sederhana: masalah, pengguna, alur utama, data yang diperlukan, aturan akses, kondisi gagal, dan cara menguji. Minta AI membaca spesifikasi dan menyebutkan asumsi yang belum jelas sebelum menulis perubahan.

4.2 Kerjakan langkah kecil

Mintalah satu perubahan yang dapat diperiksa. Contohnya: membuat formulir dengan tiga kolom, menambahkan validasi untuk satu kolom, atau menampilkan pesan ketika penyimpanan gagal. Setelah perubahan diuji, catat hasilnya sebelum melanjutkan.

Permintaan “bangun seluruh aplikasi lengkap dengan login, pembayaran, laporan, dan notifikasi” sulit diperiksa. Jika hasilnya bermasalah, Anda tidak tahu bagian mana yang menyebabkannya. Langkah kecil memperkecil wilayah kesalahan dan membuat pembatalan lebih mudah.

4.3 Berikan konteks yang cukup

Konteks yang berguna meliputi tujuan proyek, struktur yang ada, versi alat yang diketahui, batasan desain, perilaku yang diinginkan, dan pesan error lengkap. Berikan hanya informasi yang diperlukan. Jangan menempelkan rahasia, kata sandi, token, kunci layanan, data pribadi pelanggan, atau salinan database ke chat AI publik.

4.4 Contoh prompt buruk dan prompt lebih baik

Prompt	Masalah atau kelebihan
“Buat aplikasi stok yang sempurna.”	Terlalu kabur; tidak menjelaskan pengguna, aturan stok, data, atau ukuran keberhasilan.
“Tambahkan formulir barang pada halaman inventaris. Field wajib: nama dan jumlah. Jumlah harus bilangan bulat tidak negatif. Jelaskan berkas yang akan diubah, asumsi, dan cara menguji. Jangan mengubah login atau database sebelum saya menyetujui.”	Lebih baik karena ruang lingkup, aturan, dan batas perubahan terlihat.
“Perbaiki error ini.”	Tidak cukup konteks; AI perlu pesan error lengkap, tindakan yang memunculkannya, dan perubahan terakhir.
“Saat menyimpan jumlah 0, aplikasi menampilkan berhasil tetapi data tidak berubah. Tunjukkan kemungkinan penyebab, pemeriksaan yang perlu dilakukan, dan jangan menyimpulkan sebelum ada bukti.”	Mendorong diagnosis bertahap dan memisahkan dugaan dari fakta.

4.5 Minta penjelasan, bukan hanya hasil

Minta AI menyebutkan berkas yang diubah, alasan perubahan, asumsi, risiko, dan langkah untuk membatalkannya. Minta penjelasan dengan kata sederhana. Jika AI tidak dapat menjelaskan mengapa sebuah aturan diperlukan, jangan langsung menerimanya.

Pertanyaan yang dapat dipakai: “Apa yang berubah?”, “Apa yang tidak berubah?”, “Apa yang dapat rusak?”, “Bagaimana menguji kasus gagal?”, dan “Bagian mana yang masih berupa dugaan?”

4.6 Catatan proyek

Sediakan catatan yang berisi tujuan, keputusan, struktur data, akun penguji, perubahan terakhir, hasil pengujian, error terbuka, dan hal yang belum diuji. Catatan tersebut mencegah AI dan manusia mengulang asumsi yang sama. Jangan menyimpan rahasia di catatan proyek yang dibagikan.

4.7 Menggunakan lebih dari satu AI

Gunakan lebih dari satu AI hanya jika terbukti membantu, misalnya satu alat membantu menjelaskan dan alat lain membantu menantang asumsi. Jangan menyalin jawaban antaralat tanpa verifikasi. Banyak jawaban yang sama bukan bukti bahwa jawaban tersebut benar.

4.8 Prompt nyata dan pengalaman penulis

> **Status pengetahuan: Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis)** untuk penggunaan file `agen.md`, custom instructions, dan skill sebagai panduan kerja AI. Pengaruh masing-masing unsur terhadap mutu jawaban AI belum saya ukur secara terpisah, sehingga perbandingan efektivitasnya belum diuji.

Gaya prompt saya. Saya tidak memakai rumus kalimat baku atau istilah teknis yang tinggi. Saya menulis apa adanya, dengan bahasa sehari-hari yang saya pahami, seperti mengobrol dengan rekan kerja: apa yang saya butuhkan atau masalah apa yang sedang muncul di layar.

Cara menjaga arah AI. Karena prompt saya tidak baku, saya tidak bergantung pada prompt saja. Di repositori proyek, saya mencantumkan file `agen.md` sebagai acuan kerja menyeluruh. File ini disusun dengan bantuan asisten AI (Claude), dan berisi konteks proyek, tujuan akhir aplikasi, aturan kerja, keputusan penting, serta hal-hal yang tidak boleh diubah sembarangan. Dokumen seperti ini perlu dibaca oleh asisten AI

yang membantu proyek, agar asisten memahami proyek secara utuh dan mengikuti arah yang sudah ditetapkan dari awal sampai akhir.

Selain itu, saya memberikan instruksi khusus (custom instructions) pada pengaturan AI dan menggunakan skill yang relevan untuk mengarahkan cara kerja asisten. Keduanya juga disusun dengan bantuan asisten AI. Keduanya bukan pengganti pemeriksaan manusia. Fungsinya menjaga konteks, aturan, dan kebiasaan kerja agar tidak hilang ketika proyek dikerjakan bertahap atau ketika percakapan dengan AI berganti.

Dokumen acuan, custom instructions, dan skill harus diperlakukan sebagai alat bantu pengarah, bukan bukti bahwa hasil AI selalu benar. Setiap perubahan tetap perlu dibaca, diuji, dicatat, dan dibandingkan dengan kebutuhan proyek. Jika isi dokumen acuan sudah tidak sesuai, perbarui dulu sebelum dijadikan dasar perubahan berikutnya.

Contoh dari praktik saya (diringkas, bukan kutipan persis). Ketika tampilan aplikasi GAS menyisakan ruang kosong di bawah layar, saya menjelaskan keluhan itu langsung kepada AI. Lalu kami berdiskusi memindahkan tampilan depan ke Vercel dan GitHub, sambil tetap membaca data dari Google Sheets, dengan selalu mengacu pada file `agen.md` di repositori proyek.

Pelajaran. Bahasa sehari-hari sudah cukup, asalkan asisten AI mempunyai dokumen acuan yang benar dan hasilnya tetap saya periksa, sebagaimana dijelaskan di Bab 5.

4.9 Latihan menulis prompt

Tulis prompt untuk satu perubahan kecil dengan susunan: konteks, tujuan, batasan, data contoh yang tidak sensitif, permintaan penjelasan, dan cara pengujian. Tandai bagian yang merupakan fakta, asumsi, serta pertanyaan terbuka. Jangan memasukkan data pelanggan asli.

Bab 5 — Memeriksa dan Mencurigai Hasil AI

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis)** berlaku untuk temuan lapangan di bagian 5.11 (proyek POS, tampilan GAS, dan otomatisasi pemeriksaan). Uraian pemeriksaan lainnya adalah panduan belajar dan bukan bukti bahwa seluruh metode telah diuji oleh penulis.

5.1 Mengapa hasil AI perlu dicurigai?

AI dapat menghasilkan jawaban yang terdengar yakin tetapi salah. Ia dapat mengarang fungsi atau paket, memakai cara lama, salah memahami logika usaha, atau menulis perubahan yang cocok dengan contoh kecil tetapi gagal pada kondisi nyata. Sikap mencurigai bukan berarti menolak semua bantuan AI; sikap ini berarti meminta bukti sebelum mempercayai hasil.

5.2 Jenis kesalahan yang sering muncul

- **Halusinasi fungsi atau paket.** AI menyebut nama fungsi, paket, atau pengaturan yang tidak tersedia pada versi proyek.
- **Cara lama.** Saran mungkin pernah benar, tetapi tidak lagi cocok dengan dokumentasi atau versi yang digunakan.
- **Logika keliru.** Tampilan dan alur terlihat benar, tetapi aturan bisnisnya salah, misalnya stok dapat menjadi minus atau diskon diterapkan dua kali.
- **Perubahan terlalu luas.** Permintaan kecil mengubah berkas yang tidak terkait dan menimbulkan masalah baru.

5.3 Pemeriksaan dokumentasi dan versi

Periksa dokumentasi resmi alat yang digunakan. Cocokkan nama fungsi, cara konfigurasi, batasan, dan versi. Jika sumber resmi tidak mendukung

jawaban AI, tandai sebagai belum terbukti. Jangan menganggap hasil pencarian, komentar forum, atau jawaban AI sebagai sumber tunggal untuk keputusan berisiko.

5.4 Baca perubahan sebelum menjalankan

Bandingkan berkas sebelum dan sesudah perubahan. Cari perubahan pada login, hak akses, koneksi database, aturan perhitungan, dan konfigurasi. Tanyakan apakah ada bagian yang tidak diminta ikut berubah. Simpan titik aman sebelum perubahan besar agar Anda dapat kembali bila perlu.

5.5 Typecheck, lint, tes, dan uji manual

Pemeriksaan otomatis dapat menemukan sebagian masalah, tetapi tidak menjamin aturan usaha benar. *Typecheck* membantu menemukan ketidaksesuaian tipe. *Lint* membantu menemukan pola penulisan bermasalah. Tes otomatis memeriksa skenario yang telah ditulis. Uji manual memeriksa pengalaman nyata dari sudut pandang pengguna.

Gunakan kombinasi tersebut. Jika pemeriksaan otomatis berhasil, tulis "pemeriksaan otomatis berhasil pada kondisi yang diuji", bukan "aplikasi pasti benar". Jika belum menjalankan tes, tulis "belum diuji".

5.6 Kasus uang dan persediaan

Periksa diskon, pajak, pembulatan, nilai nol, angka negatif, mata uang, stok kosong, stok minus, pengembalian, dan transaksi yang dikirim dua kali. Tentukan aturan sebelum meminta AI menulisnya. Contoh ilustrasi: jika harga dan diskon menghasilkan pecahan, siapa yang menentukan pembulatan dan pada tahap mana pembulatan dilakukan? Jawaban itu adalah keputusan usaha, bukan keputusan otomatis AI.

5.7 Tampil benar tidak sama dengan benar

Layar dapat menampilkan pesan "berhasil" walaupun penyimpanan gagal. Tombol dapat terlihat hanya untuk pemilik tetapi jalur data masih menerima permintaan dari pengguna biasa. Laporan dapat tampak rapi

tetapi mengambil rentang tanggal yang salah. Uji hasil akhir dan jalur di belakangnya.

5.8 Laporan yang jujur

Pisahkan laporan menjadi tiga bagian: **fakta** yang dapat diamati, **dugaan** yang masih perlu dibuktikan, dan **belum diuji** yang belum memiliki hasil. Format sederhana ini membantu tim menghindari kesimpulan terlalu cepat.

5.9 Daftar tanda bahaya

Gunakan checklist lengkap di Lampiran C. Tanda bahaya antara lain: jawaban sangat yakin tanpa sumber, nama paket tidak ditemukan, perubahan terlalu banyak, rahasia muncul dalam berkas, pesan berhasil tanpa bukti penyimpanan, aturan hak akses hanya ada di tampilan, error diselesaikan dengan menonaktifkan pemeriksaan, dan tidak ada cara untuk membatalkan perubahan.

5.10 Latihan pemeriksaan

Ambil satu perubahan kecil dari proyek latihan. Tulis apa yang berubah, apa yang diminta, apa yang tidak diminta, hasil typecheck atau lint bila tersedia, skenario manual yang dicoba, dan bagian yang belum diuji. Jika menemukan perbedaan, jangan langsung meminta AI menutupi gejalanya; cari penyebabnya.

5.11 Temuan lapangan dan pelajaran

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis).** Temuan berikut adalah pengalaman penulis yang diamati dalam penggunaan proyek, bukan bukti bahwa setiap saran AI selalu benar.

1. **Terlalu banyak berkas.** Pada proyek POS, AI ikut menghasilkan banyak berkas dan fitur yang tidak relevan dengan kebutuhan di lapangan. Saya baru menyadarinya setelah meninjau ulang proyek, lalu memangkas puluhan berkas.

2. **Saran yang kelemahannya baru terlihat setelah dicoba.** Membangun aplikasi web langsung di dalam GAS tampak masuk akal. Setelah saya mencobanya sendiri, tampilan akhirnya memunculkan iframe dan ruang kosong yang lebar di bagian bawah layar. Kelemahan itu tidak terlihat dari penjelasan AI, tetapi terlihat dari pemakaian nyata.
3. **Otomatisasi berlebihan.** AI pernah membuatkan sekitar empat atau lima pemeriksaan otomatis di GitHub yang semuanya berjalan setiap kali aplikasi diterbitkan, sampai kuota layanan saat itu habis (cerita lengkap di Pengantar). Saya menonaktifkan semuanya, dan proyek berlanjut.

Pelajaran. Saran AI yang masuk akal di atas kertas tetap harus diuji pada tampilan dan pemakaian sebenarnya.

Bab 6 — Keamanan Dasar dan Data Pelanggan

> **Belum Diuji / Rencana.** Bab ini adalah panduan kesadaran dan pemeriksaan dasar, bukan hasil audit keamanan. Penggunaan operasional yang disebutkan di bagian lain tidak membuktikan bahwa seluruh pengamanan, hak akses, atau pemulihan telah diuji.

6.1 Batas pembahasan

Bab ini membahas kesadaran dasar, bukan audit keamanan dan bukan nasihat hukum. Keamanan dasar bukan jaminan bebas dari semua serangan. Untuk sistem berisiko tinggi, data sensitif dalam skala besar, atau kewajiban hukum yang rumit, minta pemeriksaan profesional dan nasihat dari pihak yang berwenang.

6.2 Rahasia di lingkungan proyek

Kunci layanan, token, kata sandi, dan nilai rahasia lain tidak boleh ditulis langsung dalam berkas tampilan, chat publik, tangkapan layar, atau repository. Gunakan pengaturan rahasia yang sesuai dengan lingkungan kerja. Periksa juga riwayat perubahan karena rahasia yang pernah tersimpan dapat tetap terlihat dalam riwayat.

Jika kunci bocor: hentikan penggunaan kunci tersebut, cabut atau ganti dari penyedia layanan, periksa aktivitas yang mungkin terjadi, perbarui tempat penyimpanan yang benar, dan catat waktu serta tindakan yang dilakukan. Jangan menganggap menghapus satu baris sudah cukup.

6.3 Login dan hak akses

Login membuktikan siapa pengguna; hak akses menentukan apa yang boleh dilakukan. Keduanya berbeda. Uji pengguna biasa, pemilik, staf, dan akun yang dinonaktifkan. Periksa akses dari tampilan dan dari jalur permintaan data, bukan hanya dengan menyembunyikan tombol.

6.4 Memisahkan data antar toko atau pengguna

Jika aplikasi melayani beberapa toko atau organisasi, setiap permintaan data harus memiliki pembatas yang benar. Pengguna dari Toko A tidak boleh melihat data Toko B hanya karena mengetahui alamat halaman atau mengubah parameter. Uji dengan dua akun penguji dan data yang sengaja dibedakan.

6.5 Validasi input

Validasi memeriksa apakah masukan memiliki bentuk, ukuran, dan nilai yang diizinkan. Periksa di tampilan untuk membantu pengguna, tetapi jangan hanya mengandalkan tampilan. Jalur pemrosesan juga perlu menolak nilai yang tidak sah. Tulis pesan error yang membantu tanpa membocorkan rahasia atau detail internal.

6.6 Penyimpanan kata sandi

Jangan menyimpan kata sandi sebagai teks biasa. Gunakan mekanisme autentikasi yang dirancang untuk menangani kata sandi, ikuti dokumentasi resmi penyedia, dan jangan membuat sistem autentikasi sendiri tanpa pemahaman yang cukup. Jika ragu, minta bantuan profesional.

6.7 Minimalkan data pelanggan

Kumpulkan hanya data yang diperlukan untuk tujuan yang jelas. Tentukan siapa yang memerlukan akses, berapa lama data disimpan, bagaimana data diekspor atau dihapus, dan apa yang dilakukan ketika pelanggan meminta penjelasan. Jangan memasukkan data asli ke chat AI

publik. Gunakan data contoh yang dibuat khusus atau data yang telah dianonimkan dengan benar.

6.8 Kesadaran dasar tentang pelindungan data pribadi

Regulasi yang relevan di Indonesia adalah Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi (UU PDP). Rujukannya tersedia di JDIH BPK RI: https://peraturan.bpk.go.id/Details/229798/uu-no-27-tahun-2022.

Dalam Putusan Mahkamah Konstitusi Nomor 151/PUU-XXII/2024, yang diucapkan pada 30 Juli 2025, Mahkamah memaknai kata "dan" dalam Pasal 53 ayat (1) huruf b UU PDP sebagai "dan/atau". Pasal tersebut mengatur keadaan yang mewajibkan penunjukan pejabat atau petugas pelindungan data pribadi. **Dampak praktisnya:** kriteria dalam Pasal 53 ayat (1) tidak boleh diasumsikan harus terpenuhi sekaligus (kumulatif). Apakah usaha Anda terkena kewajiban tersebut perlu dinilai sesuai keadaan dan sumber hukum resmi.

Status aturan turunan UU PDP perlu diverifikasi di JDIH atau sumber pemerintah resmi terbaru sebelum keputusan hukum dibuat. Buku ini tidak menyatakan bahwa seluruh aturan turunan sudah atau belum berlaku. Bagian ini bukan nasihat hukum; konsultasikan dengan pihak yang kompeten.

6.9 Catatan teknis per platform

> **Contoh Ilustrasi untuk Belajar.** Materi berikut adalah panduan pemeriksaan, bukan bukti bahwa konfigurasi penulis telah aman atau telah diuji menyeluruh.

Google Apps Script web app. Periksa siapa yang boleh mengakses web app ("Who has access") dan identitas yang digunakan untuk menjalankan skrip ("Execute as"). Aplikasi yang berjalan atas identitas penerbit memakai kewenangan identitas tersebut, sehingga akses yang terlalu luas dapat membuka risiko penyalahgunaan. Uji akses dan otorisasi sesuai kebutuhan aplikasi.

Supabase. Aktifkan dan uji Row Level Security (RLS), kebijakan akses (*policy*), dan izin tabel yang relevan. Untuk pemisahan data antartoko, uji bahwa pengguna satu toko tidak dapat membaca atau mengubah data toko lain. Kunci *publishable* atau *anon* bukan rahasia dan bukan pengganti policy akses. Kunci *secret* atau *service_role* hanya untuk lingkungan tepercaya dan tidak boleh dikirim ke browser.

Next.js. Variabel berawalan `NEXT_PUBLIC_` dapat dimasukkan ke kode yang dikirim ke browser dan tidak boleh dipakai untuk menyimpan rahasia. Periksa juga konfigurasi aplikasi, karena nilai yang dimasukkan melalui `next.config.js` dapat ikut masuk ke bundle. Uji hasil konfigurasi pada proyek masing-masing.

6.10 Status keamanan aplikasi penulis

> **Belum Diuji / Rencana.** Pada POS dan Aplikasi Stok yang diceritakan dalam buku ini, pembatasan pekerjaan staf yang dijelaskan penulis adalah aturan operasional. Buku ini belum mendokumentasikan dan belum membuktikan pengujian teknis menyeluruh atas autentikasi, otorisasi, pembatasan hak akses, atau perlindungan endpoint. Karena itu, pembaca tidak boleh menganggap aturan kerja staf sebagai pengganti kontrol teknis.

6.11 Checklist sebelum rilis

Gunakan checklist di Lampiran B. Jangan menjadikan semua kotak centang sebagai formalitas. Setiap centang sebaiknya memiliki bukti berupa hasil uji, tautan dokumentasi, catatan keputusan, atau identitas orang yang memeriksa.

> **Berhenti sebelum rilis.** Jika rahasia pernah bocor, data antar pengguna belum terpisah, akun pemilik belum jelas, atau tidak ada cara memulihkan data, tunda rilis dan cari bantuan yang sesuai.

Bab 7 — Version Control dan Cadangan

> **Belum Diuji / Rencana.** Prosedur version control, pencadangan, dan pemulihan dalam bab ini adalah panduan untuk disiapkan dan diuji pada proyek masing-masing. Cadangan dan rollback tidak dianggap terbukti sebelum pemulihannya dicatat.

7.1 Git dan GitHub dengan perumpamaan

Git dapat dibayangkan sebagai buku riwayat perubahan proyek. Setiap titik yang disimpan menjelaskan keadaan proyek pada waktu tertentu. GitHub adalah tempat daring untuk menyimpan dan berbagi salinan repository serta meninjau perubahan. Keduanya membantu, tetapi GitHub bukan otomatis cadangan lengkap untuk seluruh data aplikasi.

7.2 Buat titik aman sebelum perubahan besar

Sebelum menambah fitur besar, mengubah struktur data, mengganti layanan, atau memperbarui banyak paket, simpan keadaan yang dapat dikenali. Beri catatan tentang apa yang bekerja, apa yang belum diuji, dan cara kembali. Jangan membuat banyak perubahan berbeda tanpa titik pemisah karena penyebab masalah akan sulit dilacak.

7.3 Cabang untuk pekerjaan terpisah

Cabang (*branch*) adalah ruang kerja terpisah dalam riwayat proyek. Gunakan cabang untuk perubahan yang dapat diperiksa secara terpisah, misalnya fitur laporan atau perbaikan tampilan. Tinjau perubahan sebelum menggabungkannya ke jalur utama. Nama cabang sebaiknya menjelaskan tujuan, bukan hanya tanggal.

7.4 Rollback langkah demi langkah

1. Catat gejala, waktu mulai, pengguna yang terdampak, dan perubahan terakhir.
2. Hentikan perubahan tambahan agar keadaan tidak makin sulit dibaca.
3. Tentukan titik aman terakhir yang memang telah diperiksa.
4. Pulihkan aplikasi atau ubah kembali bagian yang bermasalah sesuai prosedur proyek.
5. Uji alur utama, login, penyimpanan, dan data yang terdampak.
6. Catat hasil pemulihan dan rencana perbaikan permanen.

Rollback aplikasi tidak selalu mengembalikan data yang sudah berubah. Karena itu, pemulihan kode dan pemulihan data perlu direncanakan sebagai dua hal yang berhubungan tetapi berbeda.

7.5 Cadangan data terpisah dari kode

Repository menyimpan bahan dan riwayat proyek, bukan selalu data operasional, berkas unggahan, pengaturan layanan, atau konfigurasi platform. Buat cadangan data terpisah, atur aksesnya, simpan pada lokasi yang tidak mudah ikut terhapus, dan uji pemulihannya. Cadangan yang belum pernah dipulihkan masih merupakan asumsi, bukan bukti.

7.6 Yang tidak boleh masuk repository

- Kata sandi, token, kunci layanan, dan file rahasia.
- Data pribadi pelanggan atau ekspor database nyata tanpa perlindungan yang sesuai.
- Cadangan yang tidak memiliki alasan untuk dibagikan atau tidak memiliki pengaturan akses yang benar.
- Berkas sementara yang dapat membingungkan proses penerbitan atau membuka informasi internal.

7.7 Latihan pemulihan

Gunakan proyek latihan dan data contoh. Buat satu titik aman, lakukan perubahan kecil, lalu simulasikan kegagalan. Tulis langkah

kembali tanpa menghapus data nyata. Setelah itu, catat bagian yang membingungkan dan perbaiki dokumentasi sebelum proyek digunakan oleh orang lain.

Bab 8 — Biaya dan Pilihan Penyimpanan Data

> **Status pengetahuan:** bagian 8.2 dan 8.4 memuat catatan **Sudah Dipakai di Lapangan**; bagian lain adalah **Contoh Ilustrasi untuk Belajar**. Semua harga, kuota, dan ketentuan layanan dapat berubah (lihat Catatan Penting di awal buku).

8.1 Biaya bukan hanya tagihan bulanan

Sumber biaya dapat mencakup domain, hosting, database, penyimpanan berkas, pengiriman email, layanan pihak ketiga, pemrosesan, cadangan, dukungan, migrasi, dan waktu orang yang memelihara. Paket gratis tetap memiliki batas penggunaan dan dapat berubah syaratnya.

Periksa juga apakah paket gratis mengizinkan penggunaan untuk keperluan usaha (komersial). Sebagian layanan membatasi paket gratisnya untuk penggunaan pribadi atau nonkomersial. Baca ketentuan layanan dan kebijakan penggunaan wajar (*fair use*) dari penyedia, lalu catat hasilnya pada tanggal keputusan.

8.2 Pantau pemakaian dan buat batas

Tentukan siapa yang menerima pemberitahuan penggunaan, kapan pemakaian diperiksa, dan tindakan ketika mendekati batas. Catat apakah batas tersebut menghentikan layanan, menambah biaya, memperlambat proses, atau mengharuskan peningkatan paket. Jangan menunggu tagihan mengejutkan untuk pertama kali membaca ketentuan.

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis).** Pada periode penggunaan tersebut, penulis memakai paket gratis GitHub, Vercel, dan Supabase tanpa kendala, kecuali satu proyek yang mengalami insiden kuota pemeriksaan otomatis GitHub Actions habis (lihat Pengantar).

> Pelajarannya: biaya dan batas dapat datang dari hal yang berjalan otomatis di belakang layar, bukan hanya dari database.

8.3 Kepemilikan akun dan data

Akun sebaiknya berada pada identitas usaha atau klien yang akan memiliki sistem, bukan hanya pada akun pribadi pembuat. Pastikan pemilik memiliki akses yang benar, metode pemulihan, dan catatan layanan. Pastikan data dapat diekspor dalam bentuk yang dapat digunakan jika perlu berpindah penyedia.

8.4 Google Sheets + GAS

Google Sheets + GAS cocok untuk data kecil, pengguna sedikit, dan laporan sederhana. Keuntungannya adalah banyak orang telah memahami lembar kerja dan proses awal dapat terasa ringan. Namun, batasnya mencakup kuota harian, penulisan bersamaan, dan pengaturan akses yang terbatas. Ketika banyak pengguna mengubah data bersamaan atau aturan akses makin rinci, pendekatan ini perlu ditinjau kembali.

> **Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis).** Memakai GAS langsung sebagai penampil halaman web mempunyai keterbatasan tampilan berupa iframe dan ruang kosong di bagian bawah layar. Karena itu, pada Aplikasi Stok Bakso Len saya memakai pola hibrida: GitHub dan Vercel untuk tampilan depan, sedangkan Google Sheets dan GAS menjadi jalur penyimpanan data. Pola ini sudah dipakai staf di lapangan, tetapi belum 100% final.

> **Belum Diuji / Rencana.** Aplikasi stok saya membaca seluruh lembar pada setiap transaksi. Bila baris data sudah mencapai puluhan ribu, diperlukan pengarsipan tahunan. Saya belum menguji aplikasi pada jumlah baris sebesar itu.

8.5 Database cloud seperti Supabase

Database cloud cocok untuk banyak pengguna, data terstruktur, dan aturan akses rinci. Ia dapat mendukung pemisahan data dan pola

pencarian yang lebih sesuai untuk aplikasi. Batasnya: biaya dapat naik, paket gratis dapat berubah, dan aturan akses perlu dipahami dengan serius. Menggunakan database cloud tidak otomatis membuat aplikasi aman atau bebas biaya.

Catatan belajar tentang batas layanan (*Contoh Ilustrasi untuk Belajar*). Paket dan layanan database cloud dapat memiliki batas aktivitas, penggunaan, dan biaya yang bergantung pada pemakaian. Misalnya, proyek pada paket gratis dapat dijeda bila tidak aktif. Periksa kebijakan terbaru penyedia. Penulis tidak mengklaim pernah mengalami penjedaan pada proyek Supabase miliknya.

Pertimbangan	Google Sheets + GAS	Database cloud seperti Supabase
Skala awal	Data kecil dan pengguna sedikit.	Banyak pengguna atau pertumbuhan data yang lebih terencana.
Struktur data	Sederhana dan cocok untuk laporan ringan.	Terstruktur dengan hubungan dan aturan yang lebih rinci.
Penulisan bersamaan	Dapat menjadi batas dan perlu diuji.	Dirancang untuk pola aplikasi yang lebih terstruktur, tetapi tetap perlu konfigurasi dan pengujian.
Hak akses	Lebih terbatas untuk kebutuhan aplikasi yang kompleks.	Dapat dibuat lebih rinci, tetapi aturan akses harus dipahami dan diuji.
Biaya	Perlu memeriksa kuota dan perubahan layanan.	Biaya dapat naik; paket gratis dapat berubah.
Pemindahan	Ekspor lembar kerja relatif mudah, tetapi struktur aplikasi perlu disiapkan.	Rencanakan ekspor, struktur, dan pemulihan sejak awal.

8.6 Cara membandingkan secara adil

Gunakan data dan skenario yang sama. Bandingkan waktu membuat, aturan akses, penulisan bersamaan, pencadangan, ekspor, pemulihan,

biaya saat pemakaian meningkat, dan kemampuan pemelihara. Jangan menyimpulkan hanya dari harga awal atau kemudahan demo.

8.7 Catatan sumber

Buku ini tidak menetapkan harga, kuota, atau batas paket layanan sebagai angka yang berlaku saat ini. Sumber resmi tercantum di Lampiran G. Sumber untuk bagian regulasi: Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi dan database peraturan BPK.

8.8 Latihan keputusan

Tulis kebutuhan data selama tiga tahap: sekarang, perkiraan pertumbuhan yang didukung bukti, dan kondisi yang memerlukan perpindahan. Untuk setiap tahap, tulis pilihan penyimpanan, risiko, cara ekspor, dan siapa yang memeriksa biaya. Jika tidak memiliki data pendukung, tandai sebagai belum diuji atau belum diketahui.

Bab 9 — Proyek Perdana dari Awal sampai Rilis

9.1 Pilih proyek kecil dan nyata

Contoh Ilustrasi untuk Belajar. Contoh proyek di bab ini adalah pencatat stok sederhana. Alurnya meniru bentuk proyek nyata penulis, yaitu Aplikasi Stok Bakso Len, tetapi susunan teknisnya berbeda: contoh bab ini memakai Next.js, Vercel, dan Supabase, sedangkan aplikasi stok penulis memakai GitHub dan Vercel untuk tampilan serta Google Sheets dan GAS untuk data. Contoh ini adalah gambaran konseptual, bukan laporan hasil pengujian penulis (lihat "Batas pengalaman" di Pengantar). Google Sheets + GAS adalah alternatif yang dibahas di Bab 3.6 dan Bab 8.

Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis). Yang benar-benar berjalan di usaha penulis: Aplikasi Stok Bakso Len di Lubuklinggau dan POS UMKM Juara di cabang Curup, dengan pola tampilan GitHub + Vercel dan data Google Sheets + GAS. Pengguna saat ini adalah staf kasir di Curup, staf stok di Lubuklinggau, dan pemilik usaha (penulis). Staf hanya mengisi pekerjaan harian dan tidak mengubah pengaturan atau data induk. Bila ada salah input atau kendala, staf melapor kepada pemilik, lalu pemilik memperbaiki data atau kode aplikasinya. Semua sudah diuji coba dan dipakai, tetapi belum tuntas 100%.
Batas status ini: tidak membuktikan keamanan menyeluruh, pengujian hak akses, pemulihan bencana, performa pada skala besar, atau kesiapan komersial (lihat Lampiran F).

9.2 Sepuluh langkah kerja

1. **Pilih masalah.** Tulis siapa yang mengalami masalah, kapan masalah terjadi, dan apa akibatnya. Hindari kalimat luas seperti "membuat sistem lengkap"; pilih satu alur yang dapat diamati.

2. **Tulis spesifikasi.** Tetapkan pengguna, data, aturan stok, kondisi gagal, hak akses, dan definisi selesai. Nyatakan asumsi, misalnya apakah satu barang memiliki satu satuan.
3. **Buat tampilan untuk HP.** Mulai dari alur yang paling sering. Pastikan tombol, label, pesan gagal, dan hasil simpan mudah dipahami pada layar kecil. Ini rancangan awal, bukan bukti aplikasi sudah siap.
4. **Hubungkan penyimpanan.** Buat struktur data minimum di Supabase sesuai dokumentasi resmi yang berlaku. Tentukan kolom, nilai wajib, dan aturan perubahan sebelum menambah laporan.
5. **Atur login dan hak akses.** Tentukan peran, uji akun berbeda, dan pastikan permintaan data juga diperiksa. Jangan menganggap menyembunyikan tombol sebagai pengamanan.
6. **Periksa hasil AI.** Baca perubahan, cocokkan dokumentasi dan versi, jalankan pemeriksaan yang tersedia, lalu uji alur berhasil dan gagal. Pisahkan fakta, dugaan, dan belum diuji.
7. **Jalankan checklist keamanan.** Gunakan Lampiran B. Pastikan rahasia tidak masuk repository, data penguji terpisah dari data nyata, dan ada rencana jika kunci bocor.
8. **Simpan di GitHub dan terbitkan melalui Vercel.** Atur kepemilikan akun, variabel lingkungan, cabang, dan titik aman. Ikuti dokumentasi resmi yang sesuai versi saat pelaksanaan.
9. **Uji oleh satu atau dua orang.** Berikan tugas yang jelas tanpa memberi petunjuk setiap langkah. Catat tempat mereka bingung, pesan yang salah dipahami, dan tindakan yang gagal.
10. **Catat temuan dan hal yang belum beres.** Jangan menutup proyek hanya karena alur utama berhasil. Tulis bug, risiko, batasan, keputusan yang ditunda, dan siapa yang bertanggung jawab atas tindak lanjut.

9.3 Contoh spesifikasi ringkas

> **Contoh Ilustrasi untuk Belajar.**

Bagian	Contoh ilustrasi
Pengguna	Staf toko dan pemilik toko. Daftar peran lengkap ditentukan per proyek.
Alur utama	Staf melihat barang, menambah perubahan jumlah, lalu memeriksa hasil tersimpan.
Aturan	Jumlah tidak boleh negatif. Staf hanya mengisi pekerjaan harian; koreksi dilakukan oleh pemilik.
Data	Nama barang, jumlah, waktu perubahan, dan identitas pelaku.
Definisi selesai	Alur utama berjalan pada data uji, error terlihat jelas, hak akses diuji, dan pemulihan tercatat.

9.4 Sebelum menyebutnya rilis

Pastikan ada pemilik akun, titik pemulihan, cadangan data yang dapat dijelaskan, daftar pengguna penguji, catatan batasan, serta prosedur manual bila aplikasi tidak tersedia. “Sudah dapat dibuka” bukan definisi rilis yang cukup.

Bab 10 — Setelah Rilis, Pantau dan Rawat

> **Belum Diuji / Rencana.** Alur pemantauan, respons insiden, dan perawatan dalam bab ini adalah rancangan panduan. Penerapannya pada proyek tertentu, termasuk prosedur manual dan pemulihan, perlu diuji serta dicatat sebelum dijadikan prosedur operasional.

10.1 Rilis adalah awal perawatan

Setelah aplikasi digunakan, masalah baru dapat muncul karena pola penggunaan, data yang lebih beragam, layanan pihak ketiga, pembaruan paket, atau perubahan konfigurasi. Buat jadwal pemantauan yang sesuai risiko, bukan hanya menunggu laporan pengguna.

10.2 Pemantauan rutin

Periksa apakah aplikasi dapat dibuka, login berfungsi, penyimpanan berhasil, data tidak tercampur, dan proses penting selesai. Periksa pemberitahuan error dan penggunaan layanan. Catat tanggal, orang yang memeriksa, hasil, dan tindak lanjut.

10.3 Saat error terjadi di jam operasional

1. Konfirmasi gejala dengan data minimum yang aman dan catat waktu kejadian.
2. Tentukan pengguna dan fungsi yang terdampak.
3. Hentikan perubahan tambahan dan lindungi data yang sedang diproses.
4. Aktifkan prosedur manual sementara bila tersedia.
5. Komunikasikan status dengan jujur: fakta, dugaan, tindakan, dan waktu pembaruan berikutnya.
6. Pulihkan dari titik aman bila risiko membesar, lalu uji alur utama.

10.4 Prosedur manual sementara

Tentukan sebelumnya bagaimana usaha tetap berjalan ketika aplikasi tidak tersedia. Gunakan formulir atau catatan yang memiliki nomor, waktu, pelaksana, dan status rekonsiliasi. Setelah aplikasi pulih, masukkan data dengan pemeriksaan ganda dan tandai sumbernya agar tidak terjadi duplikasi.

10.5 Mencari akar masalah

Jangan berhenti pada “error sudah hilang”. Tanyakan perubahan apa yang terjadi, kondisi apa yang memicu, mengapa pemeriksaan tidak menangkapnya, dan tindakan apa yang mencegah pengulangan. Bedakan perbaikan darurat dari perbaikan permanen.

10.6 Perawatan berkala

Tinjau pembaruan paket, kunci layanan, akun pengguna, kebijakan penyedia, biaya, cadangan, dan laporan pengguna. Jangan memperbarui banyak komponen sekaligus tanpa titik aman dan rencana kembali. Periksa dokumentasi resmi ketika versi berubah.

10.7 Laporan pengguna

Minta pengguna melaporkan tindakan yang dilakukan, waktu, layar yang terlihat, pesan error, dan dampaknya. Hindari meminta kata sandi atau data pribadi yang tidak perlu. Beri nomor laporan dan status agar pengguna tahu laporan tidak hilang.

10.8 Rencana cadangan manual

Tuliskan siapa yang mengambil alih, alat yang digunakan, cara menyimpan catatan, kapan rekonsiliasi dilakukan, dan siapa yang menyetujui hasil. Rencana ini perlu dilatih setidaknya pada data contoh agar tidak baru dibaca ketika keadaan darurat.

Bab 11 — Serah Terima ke Klien (Rancangan)

> **Belum Diuji / Rencana.** Sampai buku ini ditulis, penulis belum memiliki proyek yang siap dijual atau diserahterimakan secara komersial kepada pihak luar. Seluruh isi bab ini adalah rancangan prinsip dan daftar periksa persiapan, belum diuji pada klien eksternal, dan disiapkan sebagai panduan untuk saat proyek sudah siap diserahkan kepada pelaku UMKM lain.

11.1 Kepemilikan akun

Serah terima bukan hanya mengirim alamat aplikasi. Pastikan klien memiliki atau mengendalikan akun GitHub, hosting seperti Vercel, database seperti Supabase, dan domain. Catat email pemilik, metode pemulihan, peran akses, tanggal serah terima, serta layanan yang masih terhubung.

Jangan menyimpan satu-satunya akses pada akun pribadi pembuat. Jika akses harus dibantu sementara, jelaskan durasinya dan cara mencabutnya. Pemilik usaha perlu tahu akun mana yang membayar layanan dan siapa yang dapat mengubah data atau konfigurasi.

11.2 Dokumentasi minimum

- Tujuan aplikasi dan alur utama.
- Daftar akun, peran, dan cara menambah atau menonaktifkan pengguna.
- Cara menjalankan pemeriksaan dasar dan membaca error umum.
- Cara membuat serta memulihkan cadangan.
- Daftar layanan, pemilik akun, jadwal biaya, dan sumber resmi.
- Batasan yang diketahui dan daftar hal yang belum diuji.
- Kontak dukungan dan prosedur ketika aplikasi tidak tersedia.

11.3 Pelatihan pemakai dan pemilik

Pemakai perlu belajar alur kerja sehari-hari, pesan gagal, dan prosedur manual. Pemilik perlu belajar akun, biaya, akses, cadangan, pemantauan, dan keputusan kapan meminta bantuan. Minta peserta melakukan kembali langkahnya, bukan hanya menonton demonstrasi.

11.4 Batas tanggung jawab dan dukungan

Belum diputuskan. Penulis masih berfokus menguji aplikasi di usaha sendiri dan belum membuka layanan komersial atau serah terima ke pihak luar. Durasi dukungan, jam layanan, jenis perubahan yang termasuk, dan biaya tambahan akan ditentukan nanti. Jangan menyatakan dukungan "selamanya" jika tidak ada batas layanan, waktu, biaya, dan tanggung jawab yang tertulis dengan jelas.

11.5 Kapan klien perlu beralih ke programmer profesional?

Klien perlu mencari programmer profesional ketika jumlah pengguna dan dampak bisnis meningkat, sistem memproses uang atau data sensitif, integrasi menjadi kompleks, kebutuhan audit muncul, kinerja menurun, pemulihan tidak dapat diuji, atau perubahan kecil berisiko merusak banyak alur. Beralih bukan kegagalan; itu bentuk pengelolaan risiko.

11.6 Bukti serah terima

Buat catatan yang memuat barang yang diserahkan, tanggal, pihak yang menerima, akses yang telah diuji, pelatihan yang dilakukan, batas dukungan, dan hal yang masih terbuka. Minta kedua pihak memeriksa catatan tersebut. Jika ada bagian yang belum siap, tulis sebagai belum selesai, bukan disamarkan sebagai selesai.

Penutup

Mulailah dari satu proyek kecil yang benar-benar membantu pekerjaan. Tulis masalahnya, buat perubahan sedikit demi sedikit, periksa hasil AI, lindungi data, simpan titik aman, dan uji dengan jujur. Jangan mengukur kemajuan dari banyaknya fitur; ukur dari seberapa jelas Anda memahami apa yang berubah dan seberapa siap Anda memulihkan ketika sesuatu gagal.

Anda tidak perlu menguasai semua istilah sebelum memulai, tetapi Anda perlu bersedia belajar istilah yang menyentuh risiko proyek. Tanggung jawab manusia tidak dapat dialihkan hanya karena sebuah jawaban berasal dari AI. Berhenti, meminta bantuan, atau menunda rilis sering kali merupakan keputusan yang lebih baik daripada memaksakan hasil yang belum dipahami.

Tentang Komunitas dan Koreksi Naskah

Buku ini disiapkan sebagai panduan dasar bagi siapa saja yang ingin mulai belajar merakit sistem dengan jujur dan bertanggung jawab, termasuk sebagai pegangan awal bagi Komunitas Ngulik Digital yang diinisiasi oleh penulis. Karena buku ini terus disempurnakan berdasarkan pengujian lapangan, pembaca yang ingin menyampaikan koreksi, saran, atau masukan konstruktif dapat menghubungi penulis melalui email berikut atau melalui kanal resmi komunitas.

Email penulis: peyaje531@gmail.com

Pembaruan edisi, portofolio proyek, dan informasi Komunitas Ngulik Digital akan disampaikan melalui kanal resmi komunitas atau catatan pembaruan edisi berikutnya.

Lampiran A — Glosarium Istilah Teknis

Istilah	Arti sehari-hari
AI	Alat yang menghasilkan atau mengolah keluaran berdasarkan pola dari data dan instruksi.
API	Jalur komunikasi teratur antarbagian atau antarlayanan.
Back-end	Bagian belakang yang memproses aturan dan data.
Branch (cabang)	Ruang kerja terpisah untuk perubahan tertentu.
Database	Tempat menyimpan data agar dapat dicari dan diubah secara teratur.
Deploy (penerbitan)	Proses membuat aplikasi tersedia pada lingkungan yang dapat diakses.
Front-end	Bagian aplikasi yang dilihat dan digunakan pengguna.
GAS	Google Apps Script, layanan skrip Google yang dapat menghubungkan aplikasi dengan Google Sheets.
Hosting	Tempat aplikasi berjalan dan dapat diakses melalui jaringan.
Lint	Pemeriksaan pola penulisan dan kemungkinan masalah tertentu.
Repository	Arsip proyek dan riwayat perubahan yang disimpan bersama.
RLS	Row Level Security, aturan di database yang membatasi baris data yang boleh dilihat atau diubah pengguna tertentu.
Rollback	Mengembalikan aplikasi ke titik yang lebih aman.

Istilah	Arti sehari-hari
Supabase	Contoh layanan database cloud; detail layanan harus diperiksa dari sumber resmi.
Typecheck	Pemeriksaan apakah jenis data digunakan secara konsisten.
Variabel lingkungan	Nilai konfigurasi yang disediakan oleh lingkungan kerja, termasuk nilai rahasia.

Lampiran B — Checklist Keamanan Sebelum Rilis

Identitas, akun, dan rahasia

- ☐ Pemilik akun GitHub, hosting, database, dan domain sudah ditentukan.
- ☐ Kata sandi, token, dan kunci layanan tidak ditulis di proyek, chat publik, atau dokumentasi terbuka.
- ☐ Variabel lingkungan pada lingkungan rilis sudah diperiksa.
- ☐ Metode pemulihan akun dan autentikasi tambahan telah ditinjau.
- ☐ Rencana pencabutan dan penggantian kunci bila bocor sudah tertulis.

Hak akses dan data

- ☐ Peran pengguna telah ditulis dan diuji dengan akun berbeda.
- ☐ Pengguna tidak dapat melihat atau mengubah data milik toko atau pengguna lain tanpa izin.
- ☐ Input ditolak bila kosong, terlalu panjang, atau memiliki nilai yang tidak diizinkan.
- ☐ Kata sandi tidak disimpan sebagai teks biasa.
- ☐ Data pelanggan yang dikumpulkan sudah diminimalkan dan tujuannya jelas.
- ☐ Data asli tidak digunakan dalam chat AI publik.

Cadangan dan respons

- ☐ Cadangan data terpisah dari repository telah dibuat.
- ☐ Pemulihan dari cadangan pernah diuji pada data yang aman.
- ☐ Prosedur manual ketika aplikasi tidak tersedia telah ditulis.
- ☐ Batas keamanan dan hal yang belum diuji sudah dicatat.
- ☐ Aspek hukum telah ditinjau oleh pihak yang sesuai bila diperlukan; buku ini bukan nasihat hukum.

Lampiran C — Checklist Pemeriksaan Hasil AI dan Tanda Bahaya

Checklist pemeriksaan

- ☐ Tujuan perubahan tertulis dalam satu kalimat.
- ☐ Berkas yang diubah dan alasan perubahan telah dibaca.
- ☐ Tidak ada berkas atau aturan yang berubah tanpa alasan jelas.
- ☐ Nama fungsi, paket, dan cara konfigurasi cocok dengan dokumentasi resmi serta versi yang digunakan.
- ☐ Pemeriksaan otomatis yang tersedia sudah dijalankan dan hasilnya dicatat.
- ☐ Alur berhasil, gagal, kosong, duplikat, dan batas nilai telah diuji.
- ☐ Kasus uang, pembulatan, diskon, stok minus, dan pengiriman ganda telah dipertimbangkan bila relevan.
- ☐ Hak akses diuji melalui lebih dari satu peran, bukan hanya melalui tampilan.
- ☐ Titik aman dan cara rollback sudah tersedia.
- ☐ Fakta, dugaan, dan belum diuji telah dipisahkan dalam laporan.

Tanda bahaya

- AI terdengar yakin tetapi tidak dapat menyebutkan sumber atau versi.
- Paket atau fungsi yang disarankan tidak ditemukan dalam dokumentasi resmi.
- Solusi menghapus pemeriksaan untuk membuat error menghilang.
- Rahasia, data pribadi, atau konfigurasi sensitif muncul dalam perubahan.
- Perubahan menyentuh banyak area padahal permintaannya kecil.
- Pesan “berhasil” muncul tanpa bukti data tersimpan.
- Hak akses hanya disembunyikan pada tampilan.
- Tidak ada cadangan atau cara kembali yang dapat diuji.

Lampiran D — Templat Prompt untuk Belajar

> **Contoh Ilustrasi untuk Belajar.** Templat ini adalah titik awal yang dapat Anda sesuaikan.

Templat prompt perubahan kecil

- **Konteks proyek:** [Jelaskan aplikasi, pengguna, dan bagian yang relevan].
- **Tujuan:** [Jelaskan satu hasil yang ingin dicapai].
- **Batasan:** [Sebutkan bagian yang tidak boleh diubah, aturan data, dan batas risiko].
- **Data contoh:** [Gunakan data buatan, bukan data pelanggan asli].
- **Permintaan:** "Jelaskan rencana, berkas yang akan diubah, asumsi, risiko, dan cara menguji sebelum memberikan perubahan."

Templat prompt pemeriksaan

"Saya memiliki perubahan berikut: [tempel ringkasan aman]. Tolong pisahkan fakta yang terlihat, dugaan penyebab, dan hal yang belum diuji. Cocokkan saran dengan dokumentasi resmi versi [versi bila diketahui]. Jangan menyarankan menonaktifkan keamanan hanya untuk menghilangkan gejala. Berikan langkah uji manual dan cara membatalkan perubahan."

Gaya prompt penulis (*Sudah Dipakai di Lapangan*). Saya tidak memakai rumus baku. Saya menulis dengan bahasa sehari-hari, seperti mengobrol, dan mengarahkan AI melalui file `agen.md` di repositori, custom instructions, dan skill (lihat Bab 4.8). Temuan lapangan diringkas di Bab 5.11. Pengaruh dokumen acuan terhadap mutu jawaban AI belum saya catat secara terpisah; statusnya belum diuji.

Pelajaran. Instruksi tidak harus rumit. Yang menentukan adalah arah proyek yang tertulis dan pemeriksaan hasilnya.

Lampiran E — Catatan Versi Buku

Tanggal	Versi	Bagian utama	Alasan
4 November 2026	Edisi 1, versi 4.0	Seluruh buku	Menambah halaman hak cipta dan Catatan Penting tunggal (menggantikan disclaimer yang berulang); memindahkan catatan batas layanan database dari daftar kegagalan ke Bab 8; menambah Bab 3.6 (pola hibrida) dan Bab 6.10 (status keamanan aplikasi penulis); menambah pemeriksaan penggunaan komersial paket gratis; menjelaskan dampak praktis Putusan MK; merapikan penomoran, label status, Lampiran F, dan tautan Lampiran G.
4 November 2026	3.1 (pra-rilis)	Pengantar, Bab 6.8, Lampiran F–G	Batas pengalaman Supabase/Next.js, transparansi penggunaan AI, keamanan GAS/Supabase/Next.js, penjelasan Putusan MK.
4 November 2026	3 (pra-rilis)	Pengantar, Penutup, Lampiran F–G	Narasi jurnal penulis, tiga kategori status pengetahuan, sumber resmi.

Catatan versi berikutnya ditambahkan setiap kali ada perubahan yang memengaruhi langkah teknis, alat, harga, paket gratis, dokumentasi, atau aturan layanan.

Lampiran F — Daftar Terbuka Hal-Hal yang Belum Diuji Penulis

Daftar ini diperbarui berdasarkan pengujian nyata. Jangan mengubah "belum diuji" menjadi "sudah diuji" tanpa bukti dan catatan tanggal.

Sudah Dipakai di Lapangan (Pengalaman Nyata Penulis)

- Penggunaan paket gratis GitHub, Vercel, dan Supabase pada empat proyek awal (pada periode tersebut; bukan jaminan kondisi sekarang).
- Pola hibrida GitHub + Vercel (tampilan) dengan Google Sheets + GAS (data) pada Aplikasi Stok Bakso Len.
- POS UMKM Juara dipakai harian oleh staf kasir cabang Curup.
- Aplikasi Stok Bakso Len dipakai harian oleh staf stok di Lubuklinggau.
- Alur penanganan kendala: staf melapor, lalu pemilik memperbaiki aplikasi.
- Dokumen acuan, custom instructions, dan skill di repositori dipakai sebagai panduan kerja AI.

Belum Diuji / Rencana

- Uji pemulihan cadangan (POS dan Aplikasi Stok) dari salinan yang dapat digunakan. Belum ada hasil pengujian yang dicatat.
- Uji hak akses peran kasir versus pemilik, dan pemisahan data antartoko pada skenario multi-toko.
- Dokumentasi dan pengujian teknis autentikasi, otorisasi, pembatasan hak akses, serta perlindungan endpoint pada POS dan Aplikasi Stok.
- Contoh Next.js, Vercel, dan Supabase di Bab 3 dan Bab 9 sebagai satu sistem (contoh ilustrasi, bukan hasil uji penulis).

- Perilaku Google Sheets + GAS pada jumlah baris data yang sangat besar.
- Pengaruh `agen.md`, custom instructions, dan skill terhadap mutu jawaban AI, diukur secara terpisah.
- Prosedur serah terima komersial, jaminan dukungan, dan kontrak pemeliharaan untuk klien eksternal (belum ada proyek yang dijual ke pihak luar).
- Pemeriksaan ketentuan penggunaan komersial paket gratis yang dipakai penulis (lihat Bab 8.1).

Catatan sumber dan perubahan. Regulasi yang dirujuk adalah Undang-Undang Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi (database peraturan BPK, dirujuk 4 November 2026) dan Putusan MK Nomor 151/PUU-XXII/2024. Perkembangan teks, putusan, dan peraturan turunan tetap perlu diperiksa dari sumber resmi terbaru. Keputusan bisnis tentang kepemilikan akun, dukungan, dan durasi dukungan termasuk Belum Diuji / Rencana. Koreksi naskah dan masukan pembaca menjadi bahan pembaruan edisi berikutnya.

Lampiran G — Sumber Resmi dan Tanggal Rujukan

Daftar berikut adalah sumber resmi yang dirujuk dalam buku ini. Tanggal rujukan untuk Edisi 1 adalah 4 November 2026. Harga, kuota, fitur, dan ketentuan layanan dapat berubah setelah tanggal tersebut. Periksa kembali halaman resmi sebelum menerapkan langkah teknis atau mengambil keputusan biaya.

Regulasi

- JDIH BPK RI, UU Nomor 27 Tahun 2022 tentang Pelindungan Data Pribadi: https://peraturan.bpk.go.id/Details/229798/uu-no-27-tahun-2022
- Putusan Mahkamah Konstitusi Nomor 151/PUU-XXII/2024 (dibacakan 30 Juli 2025), naskah: https://s.mkri.id/simpp/ds/6889cf732767b.pdf
- Mahkamah Konstitusi RI, ringkasan berita putusan: https://www.mkri.id/berita/mk:-data-pribadi-warga-wajib-dilindungi-negara-secara-maksimal-23553

Dokumentasi teknis

- Next.js: https://nextjs.org/docs
- Variabel lingkungan Next.js: https://nextjs.org/docs/pages/guides/environment-variables
- React: https://react.dev
- Vite: https://vite.dev/guide/
- GitHub dan GitHub Actions: https://docs.github.com
- Billing GitHub Actions: https://docs.github.com/en/billing/concepts/product-billing/github-actions
- Vercel: https://vercel.com/docs
- Harga Vercel: https://vercel.com/pricing

- Supabase: https://supabase.com/docs
- Row Level Security Supabase: https://supabase.com/docs/guides/database/postgres/row-level-security
- Harga Supabase: https://supabase.com/pricing
- Google Apps Script: https://developers.google.com/apps-script
- Web app dan identitas eksekusi Google Apps Script: https://developers.google.com/apps-script/manifest/web-app-api-executable
- Kuota Google Apps Script: https://developers.google.com/apps-script/guides/services/quotas

Komunitas dan koreksi. Komunitas Ngulik Digital adalah wadah belajar bersama yang diinisiasi oleh penulis. Kritik, saran, atau koreksi konstruktif dapat dikirim ke: peyaje531@gmail.com